

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ
ФЕДЕРАЦИИ ПЕНЗЕНСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ

Кафедра «Вычислительная техника»

ОТЧЕТ

по лабораторной работе №4
по дисциплине: "Логика и основы алгоритмизации в инженерных задачах" на
тему: " Бинарное дерево поиска"

Выполнил:

Студент группы 24ВВВ3

Мингалёв М.А.

Принял:

к.т.н., доцент, Юрова О. В.

к.т.н., Деев М.В.

Пенза 2025

Цель

Изучение бинарное дерево поиска.

Лабораторное задание

Задание:

- Реализовать алгоритм поиска вводимого с клавиатуры значения в уже созданном дереве.
- Реализовать функцию подсчёта числа вхождений заданного элемента в дерево.

Результат программы

```
E:\gitnew\laba4\laba4\x64\Debug\laba4.exe
-1 - окончание построения дерева
Введите число: 2
Введите число: 10
Введите число: 2
Введите число: 45
Введите число: 43
Введите число: 45
Введите число: 6
Введите число: 2
Введите число: 10
Введите число: 45
Введите число: -1
Построение дерева окончено

Дерево:
      2
     /
    2
   /
  2
   \
   6
    \
   10
    /
   10
    \
   43
    /
   45
  /
 45
 /
45

=== ЗАДАНИЕ 1: Поиск значения в дереве ===
Введите значение для поиска: 6
Значение 6 найдено в дереве!
Элемент 6 находится на уровне 2

=== ЗАДАНИЕ 2: Подсчет числа вхождений ===
Введите значение для подсчета вхождений: 45
Число вхождений значения 45 в дереве: 3
Уровни всех найденных элементов 45:
  Найден элемент 45 на уровне 2
  Найден элемент 45 на уровне 4
  Найден элемент 45 на уровне 5
```

Рисунок 1 – бинарное дерево

Листинг

Файл 1-laba4.cpp

```
#define _CRT_SECURE_NO_WARNINGS
#include <iostream>
#include <cstdlib>
#include <locale>
#include <stdio>

using namespace std;

struct Node {
    int data;
    struct Node* left;
    struct Node* right;
};

struct Node* root = NULL;

struct Node* CreateTree(struct Node* root, struct Node* r, int data)
{
    if (r == NULL)
    {
        r = (struct Node*)malloc(sizeof(struct Node));
        if (r == NULL)
        {
            printf("Ошибка выделения памяти\n");
            exit(0);
        }

        r->left = NULL;
        r->right = NULL;
        r->data = data;
        if (root == NULL) return r;

        if (data > root->data)    root->left = r;
        else root->right = r;
        return r;
    }

    if (data > r->data)
        CreateTree(r, r->left, data);
    else
        CreateTree(r, r->right, data);

    return root;
}

void print_tree(struct Node* r, int l)
{
    if (r == NULL)
    {
        return;
    }

    print_tree(r->right, l + 1);
    for (int i = 0; i < l; i++)
    {
        printf(" ");
    }
    printf("%d\n", r->data);
    print_tree(r->left, l + 1);
}

bool searchInTree(struct Node* r, int value)
{
    if (r == NULL) {
```

```

        return false;
    }

    if (r->data == value) {
        return true;
    }

    if (value > r->data) {
        return searchInTree(r->left, value);
    }
    else {
        return searchInTree(r->right, value);
    }
}

// Функция поиска уровня элемента
int findElementLevel(struct Node* r, int value, int level)
{
    if (r == NULL) {
        return -1;
    }

    if (r->data == value) {
        return level;
    }

    if (value > r->data) {
        return findElementLevel(r->left, value, level + 1);
    }
    else {
        return findElementLevel(r->right, value, level + 1);
    }
}

// Функция подсчёта числа вхождений элемента
int countOccurrences(struct Node* r, int value)
{
    if (r == NULL) {
        return 0;
    }

    int count = 0;

    if (r->data == value) {
        count = 1;
    }

    count += countOccurrences(r->left, value);
    count += countOccurrences(r->right, value);

    return count;
}

// Функция поиска уровней всех повторяющихся элементов
void findAllElementLevels(struct Node* r, int value, int level)
{
    if (r == NULL) {
        return;
    }

    if (r->data == value) {
        printf(" Найден элемент %d на уровне %d\n", value, level);
    }

    findAllElementLevels(r->left, value, level + 1);
    findAllElementLevels(r->right, value, level + 1);
}

void freeTree(struct Node* r)

```

```

{
    if (r == NULL) return;

    freeTree(r->left);
    freeTree(r->right);
    free(r);
}

int main()
{
    setlocale(LC_CTYPE, "Russian");

    int D, start = 1;

    root = NULL;
    printf("-1 - окончание построения дерева\n");

    while (start)
    {
        printf("Введите число: ");
        scanf("%d", &D);
        if (D == -1)
        {
            printf("Построение дерева окончено\n\n");
            start = 0;
        }
        else
            root = CreateTree(root, root, D);
    }

    printf("\nДерево:\n");
    print_tree(root, 0);

    // ЗАДАНИЕ 1: Поиск значения
    printf("\n=== ЗАДАНИЕ 1: Поиск значения в дереве ===\n");
    printf("Введите значение для поиска: ");
    scanf("%d", &D);

    if (searchInTree(root, D)) {
        printf("Значение %d найдено в дереве!\n", D);

        int level = findElementLevel(root, D, 0);
        printf("Элемент %d находится на уровне %d\n", D, level);
    }
    else {
        printf("Значение %d не найдено в дереве.\n", D);
    }

    // ЗАДАНИЕ 2: Подсчет вхождений
    printf("\n=== ЗАДАНИЕ 2: Подсчет числа вхождений ===\n");
    printf("Введите значение для подсчета вхождений: ");
    scanf("%d", &D);

    int occurrences = countOccurrences(root, D);
    printf("Число вхождений значения %d в дереве: %d\n", D, occurrences);

    if (occurrences > 0) {
        printf("Уровни всех найденных элементов %d:\n", D);
        findAllElementLevels(root, D, 0);
    }

    freeTree(root);

    printf("\nНажмите Enter для выхода...");
    getchar();
    getchar();

    return 0;
}

```

Вывод

В ходе выполнения лабораторной работы были разработаны программы для выполнения заданий Лабораторной работы №4. В процессе выполнения работы были использованы знания о бинарном дереве.

Ссылка на удаленный репозиторий GitHub:

<https://github.com/MakarMn/LiOAvIZ>